Hands-On Teaching Module

Baby Robot Arm with Gamepad Control

Syllabus Summary

A hands-on robotics project where students build and control a robotic arm using a game controller and ROS2 using the QNX Real Time Operating System on a Raspberry Pi. The project teaches robotics fundamentals, embedded systems, and real-time control through practical experimentation. It provides early exposure to and familiarity with technology that is widely used in industry to build reliable embedded systems.

Project Overview

This project demonstrates how to control a 6-degree-of-freedom robotic arm using a gamepad controller and ROS2 software using QNX on a Raspberry Pi. Students learn how software, hardware, mechanical, and real-time systems work together to control physical devices.

Learning Goals

After completing this project, students should be able to:

- Understand basic robotics concepts (joints, motion, velocity)
- Use ROS2 to build simple robotic applications
- Understand basic QNX concepts and commands
- Connect typical embedded hardware (servos, gamepad, Raspberry Pi)
- Read and modify Python or C++ robotics code
- Understand real-time control concepts

- Map user input to physical motion

Target Student Level

- **High school** – Suitable for advanced students if strong guidance is provided
- **Undergrad year 1-2** – Ideal target student
- **Undergrad year 3-4** – Suitable if extend the sample to include other challenges
- **Graduate** – Maybe, depending on the use case. Could be good for grad students supervising undergrad project creation in robotics labs, using as a base project to explore AI/ML techniques with computer vision or explore digital twins, etc.

Prerequisites (Required Skills)

Students should have:

- **Basic programming.** Familiarity with Python is recommended.
- **Basic Linux command line skills.** QNX is the operating system used for the project, but because QNX is POSIX compliant, it is very similar to Linux. Students having some facility at the Linux command-line will easily adapt to QNX commands.
- **Intro to electronics (wiring, power).** Students should be comfortable about handling circuit boards and have appropriate electronics discipline (careful around static discharge, not placing boards on metal surfaces, etc). No soldering is required.

The following skills are helpful but optional, although they can be learned through the process of using the project:

- **Basic concepts of robotics are helpful but not necessary.** Concepts of kinematics and 3D space are helpful to understand how the robot works and is controlled but can be taught using the robot as an example.
- **C++ experience.** In addition to Python, C/C++ is also used in project, which could provide an opportunity to contrast the difference between “hard” vs “soft” programming languages used in embedded projects.
- **ROS2 familiarity.** The program uses two ROS2 nodes to communicate between the gamepad and the arm. If the student has no ROS exposure, they can quickly develop an understanding of essential ROS concepts using the robot arm source as a guide.
- **Embedded systems exposure.** Understanding concepts like host/target development, image creation, networking, and file transfer are useful to get the students building software on their laptops and transferring it to the Raspberry Pi.

Hardware Requirements

Minimum setup per student/team:

- Raspberry Pi 5 (8GB RAM and 32GB micro-SD suggested minimums)
- 6-DOF 3D-printed robotic arm kit
- PCA9685 servo driver (I²C based)
- USB or Bluetooth gamepad (Xbox/PS-style)
- 6V power supply for servos
- Six servos (3 x MG996R, 3 x SG90)
- Wiring + breadboard

Optional:

- Camera (for vision-based extensions)

Also, a 3D printer is necessary to print custom robot arm parts.

Software Requirements

All software is provided or free to download.

- Robot arm specific code and instructions: https://github.com/qnx/Baby-Robot-Arm-with-Gamepad-Control
- Free version of QNX SDP 8 (includes Python and C++ toolchains and build tools)
- ROS2 Humble (need to download and install on the Raspberry Pi the QNX-specific port at the link provided)

System Architecture (Simple Explanation)

The system has two main software components:

1. **Gamepad Input Node (C++)**
    - Reads controller input
    - Publishes /joy messages
2. **Arm Controller (Python)**
    - Receives /joy messages
    - Converts input into servo motion commands

Estimated Project Time

Activity	Time
Hardware assembly	3–6 hours
Software setup	2–4 hours
Running demo	1 hour
Understanding system	3–5 hours
Extensions/project	1–4 weeks

Note: If students print all mechanical parts themselves, expect an additional 20–50 hours of total printing time (approximately 1 week of elapsed time depending on printer availability). Assuming they're familiar with 3D printing software and equipment, the hands-on student effort to manage the printing is typically 5–9 hours, which includes setup and post-processing.

Suggested Course Integration

Option A: Short Module (1–2 weeks)

- Intro to robotics + ROS2
- Build and run system
- Simple modifications

Option B: Lab Series (3–6 weeks)

- Week 1: Build hardware
- Week 2: Understand control system
- Week 3: Modify mappings
- Week 4+: Add features

Option C: Full Project (term)

- Students extend functionality
- Present final demos

Example Student Activities

Beginner:

- Change joystick mapping
- Adjust speed of arm movement
- Add safety limits

Intermediate:

- Add new control modes (position vs velocity)
- Implement recording/playback of motions
- Tune servo parameters

Advanced:

- Add computer vision (camera tracking)
- Implement inverse kinematics
- Build autonomous behaviors
- Replace gamepad with web or AI control

Extension/Expansion:

- AI/ML: Train model to view and imitate user motion
- Human-computer interaction: alternative control methods
- Real-time systems: latency and responsiveness analysis
- Embedded systems: low-level hardware interfacing
- Robotics: motion planning, kinematics

Instructor FAQ

Q: Is this beginner-friendly?
Yes, with guidance. It combines hardware and software elements, so if the students don't have any embedded experience, support will be necessary.

Q: Can this project be used without QNX?
It hasn't been tested with Linux and the repo, instructions, and source target QNX. However, the concepts generically apply to Linux or other generic ROS2 setups.

Q: Do you need a 3D printer?
Currently, yes. We're investigating a solution for kits that have all the necessary pieces. However, until those are available, the parts for the robot arm must be 3D printed. They do not necessarily have to be printed by the students if the instructor has time to do that ahead of time.

Q: Is this project safe?
Yes, but beginner students should have electrical discipline and be informed about proper

handling embedded systems (exposed power pins, shocks, etc). Also, some care is needed around moving parts to avoid pinch points.